Les Finques (Pràctica MiniZinc)

Programació Declarativa, Aplicacions
Sr. Bermeo Quito, Wilber Eduardo
Dr. Mateu Villaret Auselle
16 de gener del 2026
Universitat de Girona

Miquell Coll Barneto
u1987334
u1987334@campus.udg.edu

Ramon López i Cros
u1988987
u1988987@campus.udg.edu

Índex

1 Model de dades **2**

2 Viewpoints **3**
2.1 Viewpoint 1 3
2.2 Viewpoint 2 8
2.2.1 Restriccions del model 8

3 Resultats **10**
3.1 Instàncies entrades 10
3.2 Viewpoint 1 14
3.2.1 Conclusions 15
3.2.2 Dades del sistema 16
3.3 Viewpoint 2 16
3.3.1 Comparació de solvers i heurístiques 16
3.3.2 Dades del sistema 17

4 Comentaris i reflexions **17**

1 Model de dades

Vam començar el problema abordant el model de dades, ja que seria la base per poder treballar tots dos de forma coordinada. El primer que vam fer va ser definir quines dades hauria d'introduir l'usuari i com s'haurien de tractar dins el model. D'aquí va sortir el nostre primer esquelet del model de dades (Veure figura1).

```
1 type Pos = tuple (int, int);
2
3 int: n_tipus_finques;
4 set of int: Tipus Finca 1..n_tipus_finques;
5
6% Especificació tipus de finques
7 array[Tipus Finca] of set of Pos: definicio_finques;
8
9% Dimensions de la parcel·la a edificar
10 int: height;
11 int: width;
12
13% Caselles destinades a parcs
14 set of Pos: parcs;
15
16% Caselles destinades a camins
17 set of Pos: camins;
18
19% Finques que s'han de posar, indicat el tipus que és 20 int: n_finques;
21 array[1..n_finques] of Tipus Finca: finques;
22
23% Parelles de finques que no es poden tocar
24 type Incompatibles = tuple (Tipus Finca, Tipus_Finca);
25 set of Incompatibles: incompatibles;
26
27% Finques que han de tenir com a veí una casella de parc com a minim
28 set of int: Index Finca = 1..n_finques;
29 set of Index Finca: finques amb parc;
```

Figura 1: Esquelet inicial del model de dades

En aquest disseny inicial, les coses eren força intuïtives: El taulell: Simplement dos enters per definir l'alçada i l'amplada. Parcs i camins: Els vam definir com a conjunts (SET) de coordenades (tuples fila-columna). Aquesta era la forma més ràpida de marcar les caselles ja ocupades. Les finques que s'havien de posar: Un array on cada posició indicava el tipus de finca (de l'1 al 13) que s'havia de colocar. Restriccions de veïnatge: Per a les finques que no es podien tocar, vam pensar en un conjunt de parelles (tuples amb els IDs de les finques). Finques amb parc: Per a les finques que necessitaven un parc a prop, vam definir un conjunt amb els seus índexs.

Aquest model ens servia per començar, però quan ens vam posar a pensar cadascú el seu ViewPoint, ens vam adonar d'un problema, les rotacions. Codificar les voltes i els girs de cada peça era un embolic. Aquí se'ns van ocorrer dos solucions diferents (s'explicaràn més endevant), de forma que vam encarar el model de dades esquelet a cada ViewPoint. Així doncs, hem acabat amb dos ViewPoints que són gairebé iguals en el tema de les dades d'entrada, però difereixen en la definició dels tipus de finques (Veure figures 2 i 3).

2 Viewpoints

2.1 Viewpoint 1

A l'hora de definir el ViewPoint, vaig decidir que la millor manera de representar el problema era centrar-se en la finca com un objecte sencer en lloc de veure-la com un conjunt de caselles, d'aquesta manera ens el solver s'absrteia de la cohessió entre caselles de la pròpia finca. Així doncs, el meu model es basa en tres variables per a cada finca dins el taulell: la seva posició (fila i columna) i la seva orientació específica (alguna finca té 8 orientacions, altres en tenen menys). Aquest enfocament permet que el solver no hagi de "descobrirl·la forma de la finca a cada pas, sinó que hagi de solocar una de les orientacions de la finca i prou.

Pel que fa a les orientacions, en lloc de tenir una única forma estàtica, cada finca es defineix per un conjunt de possibilitats que inclouen les rotacions de 90, 180 i 270 graus, així com les seves simetries[1]. Aquesta lògica de rotacions és la clau d'aquest model, ja que fa ue una finca encaixi segons com es giri, respectant sempre la seva forma original.

Ara bé, per organitzar tota la informació de manera que el programa la pugui llegir ràpidament, he estructurat totes les orientacions en un gran *array* de tres dimensions. Per saber quines orientacions corresponen a cada finca, es fa servir un sistema de traçabilitat basat en dos *arrays* auxiliars; un que indica on comença el bloc d'orientacions de cada tipus de finca i un altre que ens diu quantes orientacions té en total. D'aquesta manera, quan el model ha de col·locar, per exemple, la finca tipus 5, el ViewPoint té un *constraint* perquè només pugui triar entre les n orientacions definides per a aquesta peça.

Tot i que les decisions es prenen sobre la posició inicial de la finca i l'orientació d'aquesta, el taulell s'encarrega de verificar que no es solapin les finques, que no hi hagi cap cassella fora del taulell o que no es trepitgin els parcs i camins amb les finques. Així doncs, el solver actua de la següent manera: primer decideix on va una finca, i quina rotació es posa, i després deixa que les restriccions del taulell diguin si la configuració és vàlida.

[1] No totes les finques tenen simetria ni rotacions

```
include "global_cardinality.mzn";

% ----------------------------- %
% ----  DADES D'ENTRADA ------- %
% ----------------------------- %
int: alcada_taulell;
int: amplada_taulell;

int: n_finques;
array[1..n_finques] of int: finques;

int: n_parcs;
array[1..n_parcs, 1..2] of int: parcs;

int: n_camins;
array[1..n_camins, 1..2] of int: camins;

int: n_incompatibles;
array[1..n_incompatibles, 1..2] of 1..n_finques: incompatibles;

set of int: finques_amb_parc;

% ----------------------------- %
% --  DADES DE CONFIGURACIÓ --- %
% ----------------------------- %

int: n_tipus;

int: total_orientacions;
int: maxim_caselles;

array[1..n_tipus] of int: inici_orientacions;
array[1..n_tipus] of int: orientacions_finques;
array[1..n_tipus] of int: caselles_finques;

array[1..total_orientacions, 1..maxim_caselles, 1..2] of int: finques_amb_orientacions;

% ----------------------------- %
% ---  VARIABLES DE DESISIÓ --- %
% ----------------------------- %

array[1..alcada_taulell, 1..amplada_taulell] of var 0..n_finques: taulell; % Taulell

array[1..n_finques] of var 1..alcada_taulell: fila; % Fila de la casella inicial de cada finca
array[1..n_finques] of var 1..amplada_taulell: columna; % Columna de la casella inicial de cada finca
array[1..n_finques] of var 1..total_orientacions: id_orientacio; % Orientació de cada finca

% ----------------------------- %
% -------  CONSTRAINTS -------- %
% ----------------------------- %

% Com funciona?
% - Es recorren les finques que s'han de posar, i s'intenten posar SENCERES,
%     és a dir, en una cassella C del taulell ha d'anar la cassella F de la finca
%     i comprovar que totes les altres casselles no trenquen cap restricció i així les podem posar.

% Colocar les finques
constraint forall(f in 1..n_finques) (
    let {
        int: finca = finques[f], % Finca
        var int: orientacio = id_orientacio[f] % Orientació de la finca
    } in (
```

```
        orientacio >= inici_orientacions[finca] /\  % La orientació (de l'array de totes les orientacions) pertany al rang de la finca (>= primer)
        orientacio < inici_orientacions[finca] + orientacions_finques[finca] /\ % ...fins al nombre d'orientacions que té aquesta finca

        forall(casella in 1..caselles_finques[finca]) ( % Mirem que les casselles es puguin posar al taulell
            let {
                var int: fila_casella = fila[f] + finques_amb_orientacions[orientacio, casella, 1] - 1, % Agafem la fila de la cassella partint de la fila actual + l'offset del model
                var int: columna_casella = columna[f] + finques_amb_orientacions[orientacio, casella, 2] - 1 % Lo mateix amb columna
            } in (
                fila_casella in 1..alcada_taulell /\ columna_casella in 1..amplada_taulell /\ % La casella adient està dins el taulell
                taulell[fila_casella, columna_casella] = f % Hi posem la finca
            )
        )
    )
);

int: camins_o_parcs = (alcada_taulell * amplada_taulell) - sum(f in 1..n_finques)(caselles_finques[finques[f]]); % Son les que haurien de tenir parc o cami

% No solapar casselles (Global Cardinality)
constraint global_cardinality(
    [taulell[_fila, _columna] | _fila in 1..alcada_taulell, _columna in 1..amplada_taulell], % Cada cassella del taulell
    [_finca | _finca in 0..n_finques], % 0(camins o parcs) ++ Valors de finques (1, 2, 3, 4, ...) -> Lo que controlem
    [camins_o_parcs] ++ [caselles_finques[finques[_finca]] | _finca in 1..n_finques] % camins_o_parcs + caselles_finques[finques[_finca]] per a cada finca
);

% Extra per optimitzar una mica (despreciable) Evitar dualitat entre finques iguals
constraint forall(finca1, finca2 in 1..n_finques where finca1 < finca2) ( % Per a cada parella de finques
    if finques[finca1] == finques[finca2] then % La mateixa finca 2 cops
         % Eliminar la possibilitat de comptar 2 solucions amb les finques intercanviades (seria lo mateix)
        (fila[finca1] * amplada_taulell + columna[finca1]) < (fila[finca2] * amplada_taulell + columna[finca2])
    endif
);

% Parcs (no pot haver finca)
constraint forall(parc in 1..n_parcs) (
    taulell[parcs[parc, 1], parcs[parc, 2]] = 0  % Taulell[fila, columna] = 0
);

% Camins (no pot haver finca)
constraint forall(cami in 1..n_camins) (
    taulell[camins[cami, 1], camins[cami, 2]] = 0 % Taulell[fila, columna] = 0
);

% Finques que no es poden tocar
constraint forall(i in 1..n_incompatibles) ( % Per a cada parella de incompatibles (cada poarella es una fila)
    let {
        int: finca1 = incompatibles[i, 1],
        int: finca2 = incompatibles[i, 2]
```

```
    } in
    forall(_fila in 1..alcada_taulell, _columna in 1..amplada_taulell where taulell[_fila,_columna] == finca1) ( % Per a cada casella (fila, columna) de la finca en el taulell
        forall(_offset_fila, _offset_columna in -1..1 where (_offset_fila != 0 \/ _offset_columna != 0) /\ not (_offset_fila != 0 /\ _offset_columna != 0)) ( % Per a cada casella a dinstància 1 (veina) menys la cassella en si (quan ambdós són 0)

            let {
                int: _nova_fila = _fila + _offset_fila,
                int: _nova_columna = _columna + _offset_columna
            } in (
                if _nova_fila in 1..alcada_taulell /\ _nova_columna in 1..amplada_taulell then % ha de pertanyer al taulell
                    taulell[_nova_fila, _nova_columna] != finca2 % Aquí mirem diferent
                endif
            )
        )
    )
);

% Matriu booleana interna per als parcs
array[1..alcada_taulell, 1..amplada_taulell] of bool: matriu_de_parcs = array2d(1..alcada_taulell, 1..amplada_taulell, [
    exists(_parc in 1..n_parcs)(parcs[_parc,1] == _fila /\ parcs[_parc,2] == c_columna) % Existeix un parc que t'e aquesta posicio
    | _fila in 1..alcada_taulell, c_columna in 1..amplada_taulell
]);

% Finques amb parc
constraint forall(_finca in finques_amb_parc) (
    exists(_fila in 1..alcada_taulell, _columna in 1..amplada_taulell where taulell[_fila, _columna] == _finca) ( % Qualsevol de les casseles de la finca
        exists(_offset_fila, _offset_columna in -1..1 where (_offset_fila != 0 \/ _offset_columna != 0) /\ not (_offset_fila != 0 /\ _offset_columna != 0)) ( % Qualsevol casella a dinstància 1 (veina) menys la cassella en si (quan ambdós són 0)

            let {
                int: _nova_fila = _fila + _offset_fila,
                int: _nova_columna = _columna + _offset_columna
            } in (
                _nova_fila in 1..alcada_taulell /\ _nova_columna in 1..amplada_taulell /\ matriu_de_parcs[_nova_fila, _nova_columna] % Ha d'estar dins el taulell i ser un parc
            )
        )
    )
);

% ----------------------------- %
% ---------  SOLVE ------------ %
% ----------------------------- %
solve :: int_search(fila ++ columna ++ id_orientacio, first_fail, indomain_min) satisfy; % Cerca per prioritat (first_fail) orientada a finques, no a caselles (fila ++ columna ++ id_odietacio). indomain_prova el valor m'es petit del domini primer

% ----------------------------- %
% --------  OUTPUT ------------ %
% ----------------------------- %
output [
    if interador_columna = 1 then "\n" else "" endif ++
    let {
        int: valor = fix(taulell[iterador_fila,interador_columna]),
```

```
        bool: p = exists(k in 1..n_parcs)(parcs[k,1]==iterador_fila /\ parcs[k,2]==interador_columna),
        bool: c = exists(k in 1..n_camins)(camins[k,1]==iterador_fila /\ camins[k,2]==interador_columna)
    } in (
        if p then "Parc  "
        elseif c then "Cami  "
        elseif valor == 0 then ".     "
        elseif valor < 10 then show(valor) ++ "     "
        else show(valor) ++ "    " endif
    )
    | iterador_fila in 1..alcada_taulell, interador_columna in 1..amplada_taulell
] ++ ["\n"];
```

2.2 Viewpoint 2

El viewpoint que vaig escollir per modelar el problema consisteix a representar explícitament la *posició de cada casella que forma part de les finques* dins de la parcel·la global. Més concretament, cada finca es descompon en un conjunt de punts elementals, i cadascun d'aquests punts es modela com una variable que indica la seva posició lineal dins la graella.

Així, si el nombre total de quadrats que formen totes les finques és N, el model utilitza un array de variables:

$$\texttt{solucio}[1..N]$$

on cada variable pren valors en el domini $\{1, \ldots, \texttt{height} \times \texttt{width}\}$, que correspon a les posicions possibles dins la parcel·la, codificades en format lineal. Aquest viewpoint permet expressar de manera natural restriccions geomètriques, d'adjacència i de no solapament mitjançant operacions aritmètiques sobre les coordenades.

2.2.1 Restriccions del model

A continuació es descriuen les principals restriccions que garanteixen la correcció de les solucions obtingudes.

No solapament de finques La restricció global `alldifferent(solucio)` assegura que cap dos quadrats ocupats per les finques comparteixin la mateixa posició dins la parcel·la. Aquesta restricció garanteix que les finques no se solapen entre elles.

```
constraint alldifferent(solucio);
```

Col·locació correcta de les formes de les finques Cada finca té associada una definició geomètrica relativa, especificada mitjançant coordenades relatives entre els seus punts. El model imposa que, per a qualsevol parella de punts d'una mateixa finca, la distància relativa entre les seves posicions a la graella coincideixi amb alguna de les distàncies permeses segons la seva definició, considerant rotacions i simetries.

```
array[Index_Finca] of int: start = [ 1 + sum(t in 1..k-1)(npts[t]) | k in
↪  Index_Finca ];
function set of int: idxs(int: k) = start[k] .. start[k] + npts[k] - 1;

type D = tuple(int,int);

function var D: distancia(var int: a, var int: b) =
  ( abs(linearToRow(a) - linearToRow(b)),
    abs(linearToCol(a) - linearToCol(b)) );


function array[1..2] of D: possibles_distancies(int: k, int: i, int: j) =
  let {
    Tipus_Finca: t = finques[k],
    Pos: a = definicio_finques[t, i],
    Pos: b = definicio_finques[t, j],
    int: dr = abs(a.1 - b.1),
    int: dc = abs(a.2 - b.2)
  } in
    [ (dr, dc), (dc, dr) ];


```

```
function int: local_idx(int: k, int: g) = g - start[k] + 1;

constraint forall(k in Index_Finca)(
  forall(i, j in idxs(k) where i < j)(
    let {
      int: li = local_idx(k,i),
      int: lj = local_idx(k,j),
      var D: d = distancia(solucio[i], solucio[j]),
      array[1..2] of D: P = possibles_distancies(k, li, lj)
    } in
      (d == P[1]) \/ (d == P[2])
  )
);
```

Aquesta restricció garanteix que cada finca apareix a la parcel·la amb una forma vàlida i congruent amb el seu tipus.

Prohibició d'ocupar parcs i camins S'imposa que cap variable de `solucio` prengui un valor corresponent a una casella destinada a parc o camí, evitant així que s'hi construeixin finques.

```
set of int: forb = parcs union camins;
constraint forall(p in 1..N)(not (solucio[p] in forb) );
```

Restriccions d'incompatibilitat entre finques Algunes parelles de finques no poden ser veïnes. Per a aquestes parelles, el model imposa que cap punt d'una finca sigui adjacent a cap punt de l'altra finca. Aquesta restricció s'expressa mitjançant una comprovació de totes les parelles de punts implicades.

```
function var bool: adjacent4(var int: a, var int: b) =
  abs(linearToRow(a) - linearToRow(b)) + abs(linearToCol(a) - linearToCol(b)) ==
  ↪ 1;

constraint forall(m in 1..n_incompatibles)(
  let {
    int: k1 = incompatibles[m].1,
    int: k2 = incompatibles[m].2
  } in
  forall(p1 in idxs(k1), p2 in idxs(k2))(
    not adjacent4(solucio[p1], solucio[p2])
  )
);
```

Finques amb requisit de parc adjacent Per a determinades finques, s'exigeix que almenys un dels seus quadrats sigui adjacent a una casella de parc. El model expressa aquesta condició mitjançant una restricció existencial que assegura l'existència d'un punt de la finca adjacent a alguna posició de parc.

```
constraint forall(k in finques_amb_parc)(
  exists(p in idxs(k), q in parcs)(
    adjacent4(solucio[p], q)
  )
);
```

3 Resultats

3.1 Instàncies entrades

Per generar els resultats hem executat els nostres models amb 10 instàncies de diferent dificultat i amb 2 solvers diferents (i heurístiques diferents). Tot això per tal de veure com funcionen exactament els solvers i les diferències entre les heurístiques possibles.

Tot seguit, es llisten les diferents instàncies empreades(els camins en color vermell i els parcs en color groc. Les relacions es detallen amb cada figura):

Figura 2: Primera instància

Finques: [3, 5, 8, 13] Parcs: [ ] Camins: [ ] Incompatibilitats: [ ] Finques amb parc: [ ]

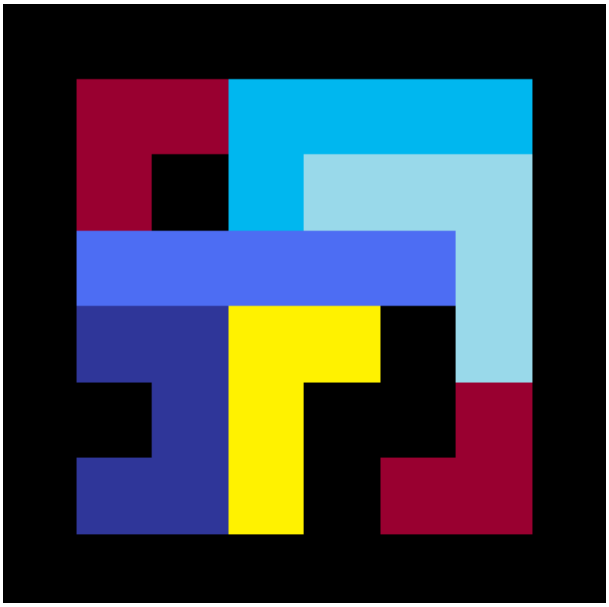

Figura 3: Segona instància

Finques: [2, 3, 7, 8] Parcs: [(1,1)(1,2)(2,1)(5,6)(6,5)(6,6)] Camins: [(4,3)(4,4)(5,3)(6,3)]

Incompatibilitats: [ ] Finques amb parc: [ ]

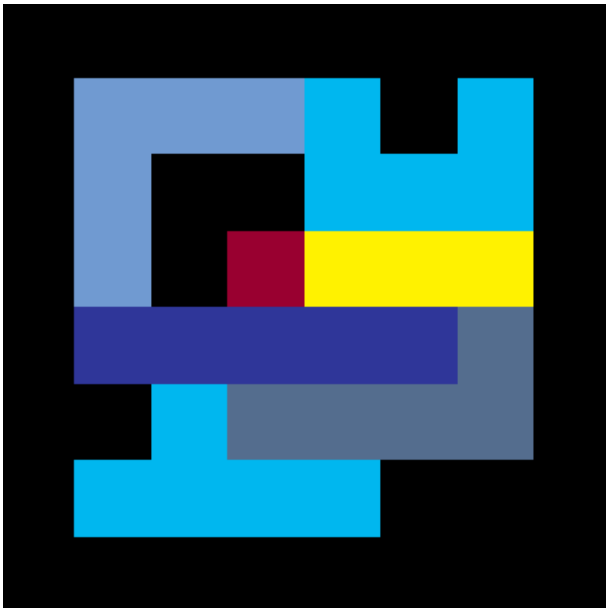

Figura 4: Tercera instància

Finques: [2, 3, 7, 8, 11] Parcs: [(3, 3)] Camins: [(3, 4)(3, 5)(3, 6)] Incompatibilitats: [ ] Finques amb parc: [1]

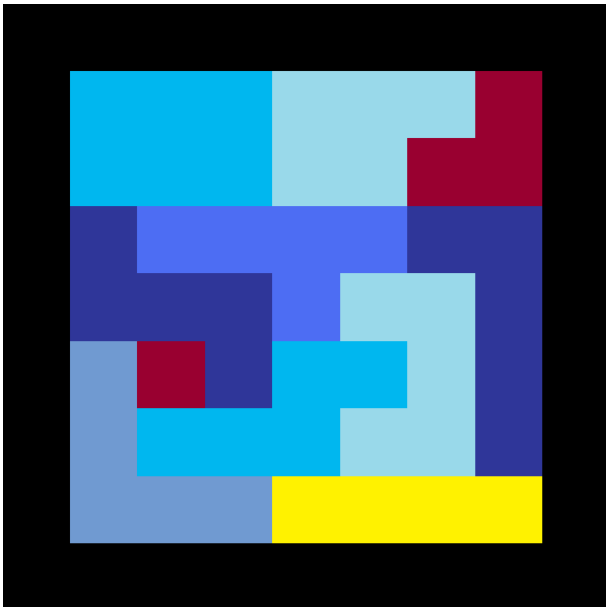

Figura 5: Quarta instància

Finques: [3, 4, 5, 7, 8, 11, 12, 13] Parcs: [(1,7)(2,6)(2,7)(5,2)] Camins: [(7,4)(7,5)(7,6)(7,7)] Incompatibilitats: [(1, 7)] Finques amb parc: [5]

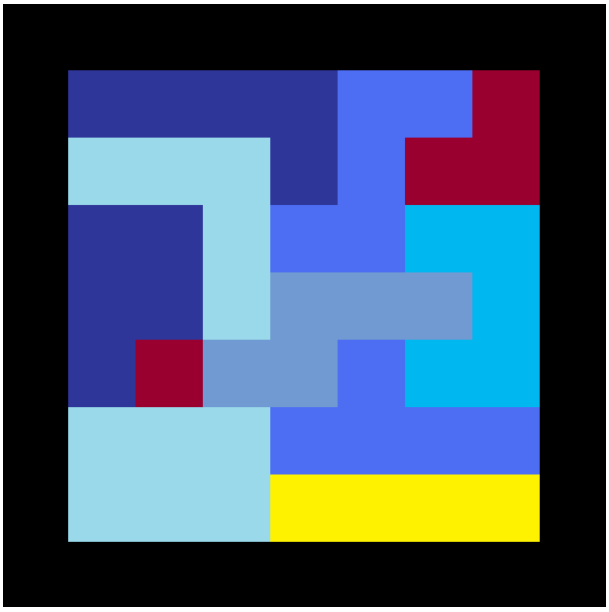

Figura 6: Cinquena instància

Finques: [3, 4, 5, 7, 8, 11, 12, 13] Parcs: [(1,7)(2,6)(2,7)(5,2)] Camins: [(7,4)(7,5)(7,6)(7,7)] Incompatibilitats: [(1, 8)] Finques amb parc: [4]

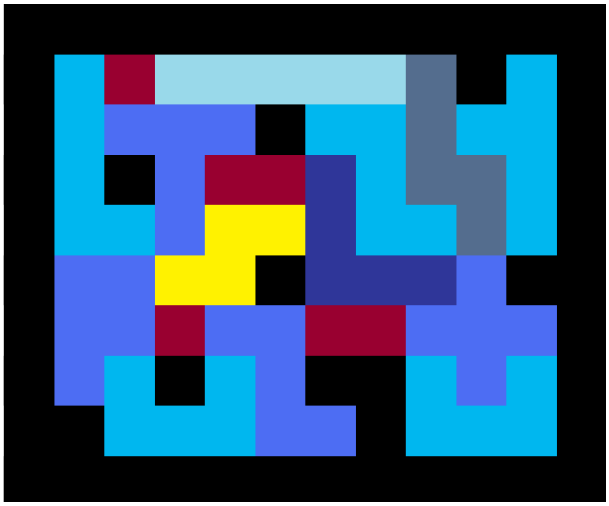

Figura 7: Sisena instància

Finques: [2,3,4,5,6,7,7, 8,10,11,12, 12] Parcs: [(3,4)(3,5)(6,3)(6,6)(6,7)(1,2)] Camins: [(4,4)(4,5)(5,3)(5,4
Incompatibilitats: [(1,8)(2,8)(3,7)] Finques amb parc: [1, 5, 9]

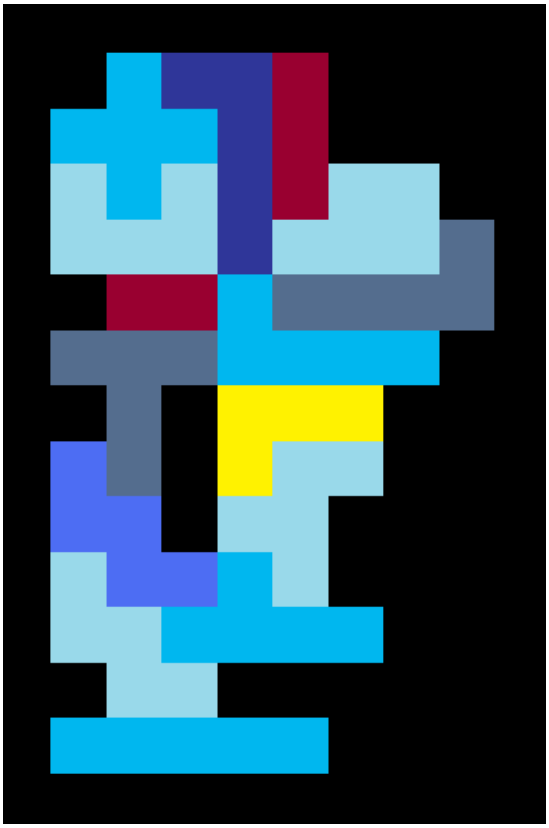

Figura 8: Setena instància

Finques: [1, 2, 3, 3, 3, 5, 6, 7, 9, 9, 10, 11] Parcs: [(1,5)(2,5)(3,5)(5,2)(5,3)] Camins: [(7,4)(7,5)(7,6)(8,4)] Incompatibilitats: [(1,7)(1,8)(1,9)(1,10)] Finques amb parc: [5, 6]

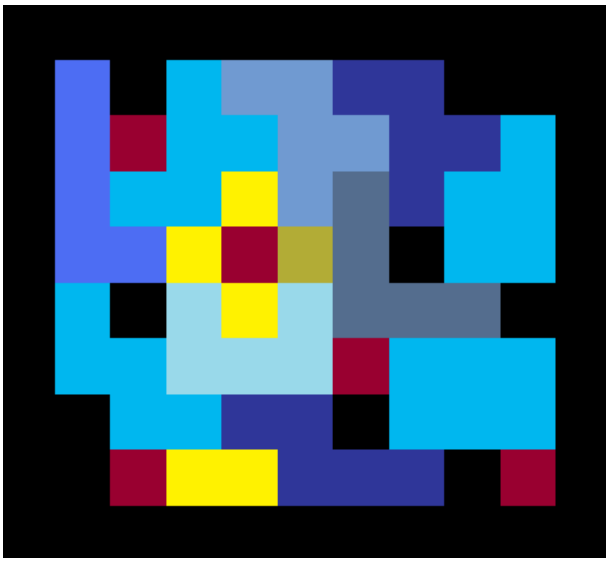

Figura 9: Vuitena instància

Finques: [1, 1, 1, 3, 4, 5, 7, 8, 9, 13] Parcs: [(2,2)(4,4)(6,6)(8,2)(8,9)] Camins: [(3,4)(4,3)(4,5)(5,4)(8,3)(8,
Incompatibilitats: [(1,6)(3,4)(9,10)(4,7)] Finques amb parc: [1,7]

Figura 10: Novena instància (UNSAT)

Finques: [5, 5, 6, 7, 8, 9, 10, 11, 13] Parcs: [(1,7)(1,8)(2,7)(2,8)(3,7)(3,8)(4,7)] Camins: [(5,8)(6,1)(6,2)(7,1)(7,2)(7,3)(8,2)] Incompatibilitats: [(1,9)(2,9)(3,7)(4,5)(6,8)(1,4)] Finques amb parc: [3, 5, 9]

Figura 11: Desena instància (UNSAT)

Finques: [1,2,3,4,5,6,7,8,9,10,11,12,13] Parcs: [(2,2)(2,8)(5,5)(8,2)(8,8)] Camins: [(1,4)(1,6)(2,4)(2,6)(3,4
Incompatibilitats: [(1,13)(2,12)(3,11)(4,10)(5,9)(6,8)] Finques amb parc: [3, 6, 9, 12]

3.2 Viewpoint 1

Instància	**Normal (No search)**	**First Fail**	**Input Order**
1 (Fàcil)	T.O.[2]	0.144s	0.177s
2	T.O.[3]	0.200s	0.188s
3	T.O.[4]	0.197s	0.297s
4	T.O.[5]	134s	312s
5	T.O.[6]	158s	20.35s
6	T.O.[7]	65s	11.799s
7	T.O.[8]	72s	4.42s
8	T.O.	642s	1.2s
9 (Difícil)	T.O.[9]	(UNSAT) 125s	(UNSAT) 125s
10 (UNSAT)	(UNSAT)	(UNSAT)	(UNSAT)

Taula 1: Temps de resolució amb Gecode 6.3.0.

[9]T.O. (Time Out): S'ha superat el límit de 15 minuts sense trobar solució.

Instància	Normal (No search)	First Fail	Input Order
1 (Facil)	0.150	0.155s	0.260s
2	0.199	0.206s	0.232s
3	0.190	2.14s	0.183s
4	3.941	47.932s	24.779s
5	7.515s	45.598s	19.463s
6	3.177s	0.624s	12.310s
7	53.24s	2.3s	4.358s
8	0.265s	0.313s	1.138s
9 (UNSAT)	(Difícil)	(UNSAT)	(UNSAT)
10 (UNSAT)	(UNSAT)	(UNSAT)	(UNSAT)

Taula 2: Temps de resolució amb Chuffed 0.13.2

3.2.1 Conclusions

Amb el Chuffed, l'anàlisi dels resultats mostra que el rendiment del model no segueix un patró lineal (com s'esperaava amb l'augment de dificultat), sinó que depèn de com cada heurística gestiona la posició i l'orientació de les finques. La heurística *first_fail* no és estable, i mostra pics de temps elevats perquè, en triar la variable amb el domini més petit, sovint prioritza decidir l'orientació (amb dominis d'1 a 8 (a vegades 1)) abans que la posició absoluta (1 a n), provocant un *backtracking* costós si la rotació escollida no encaixa geomètricament en l'espai disponible (És molt millor decidir un ordre general i llavors una orientació per a la peça, que no pas fixar la orientació com a punt inicial del problema).

Per contra, l'*input_order* és més estable en els problemes més complexes, ja que si les finques han sigut entrades per ordre de mida o dificultat (relativa a cada problema), actua com una heurística on posa la peça més complicada primer, reduint la càrrega del problema poc a poc. En quant al mode normal (sense heurística), el Chuffed demostra l'eficàcia d'aquest en aprendre sobre la marxa i no repetir problemes pasats.

Amb el Gecode, la definició d'una estratègia de cerca és absolutament indispensable. El solver entra en timeout en les primeres instàncies quan s'executa en mode "Normal"("solve satisfy", sense heurística) demostra que sense heurística el Gecode intenta decidir el valor de cada casella del taulell de forma seqüencial, i com que el taulell té $filesxcolumnes$ caselles, l'espai de cerca és immens, i el solver es perd.

Pel que fa a les seves heurístiques, sí que es veu clarament que l'*input_order* supera dràsticament el *first_fail* en els problemes més difícils (com el 8, on passem de 642s a només 1.2s). Això ocorre perquè Gecode no apren de clàusules (com ho fa el Chuffed), per tant si el *first_fail* comet l'error de fixar una orientació incorrecta, Gecode es veu obligat a explorar tota aquesta branca en va fins al final. En canvi, l'*input_order* permet seguir l'ordre de col·locació segons les dades entrades (posant les més complicades primer).

Finalment, i no tan important, la rapidesa en la detecció de problemes insatisfactibles (el darrer) confirma que l'ús del `global_cardinality` i les restriccions inicials de taulell permeten al solver descartar problemes impossibles a l'inici per evitar recorrer tota la cerca.

3.2.2 Dades del sistema

Les proves s'han realitzat en un Windows 11 i amb un Victus16d amb un intel core i7-12700H, 16BG de RAM a 4800MTUS i una 3050ti laptop. No s'ha executat per threads.

3.3 Viewpoint 2

	Geocode			Chuffed		
	baseline	ff-min	io-min	baseline	ff-min	io-min
d1	0,206	0,184	0,185	0,201	0,189	0,189
d2	1,872	0,345	0,337	0,297	0,244	0,334
d3	13,138	0,296	0,293	0,415	0,26	0,292
d4	205	TIMEOUT (600)	TIMEOUT (600)	TIMEOUT (600)	418	240
d5	209	TIMEOUT (600)	TIMEOUT (600)	TIMEOUT (600)	101	81
d6	TIMEOUT (600)	27,581	TIMEOUT (600)	TIMEOUT (600)	0,925	28,258
d7	3,683	0,885	0,813	3,421	1,144	1,187
d8	0,937	2,694	2,818	240	0,904	0,896
d9	TIMEOUT (600)	TIMEOUT (600)	TIMEOUT (600)	TIMEOUT (600)	UNSAT 1,47	TIMEOUT (600)
d10	TIMEOUT (600)	TIMEOUT (600)	TIMEOUT (600)	TIMEOUT (600)	UNSAT 120	TIMEOUT (600)

Figura 12: Taula de resolució

3.3.1 Comparació de solvers i heurístiques

En aquesta secció es comparen els resultats obtinguts utilitzant dos solvers de Constraint Programming (`Gecode` i `Chuffed`) combinats amb tres estratègies de cerca: la cerca per defecte del solver (Baseline), l'heurística *First-Fail amb valor mínim* (FF-Min) i una cerca amb *Input Order amb valor mínim* (IO-Min). S'ha establert un límit de temps de 600 segons per instància.

Resultats generals Per a les instàncies més senzilles (d1 i d2), totes les combinacions de solver i heurística aconsegueixen trobar solució en temps reduït. Tot i això, ja es pot observar que l'ús d'heurístiques explícites (especialment FF-Min) millora lleugerament els temps respecte a la cerca per defecte.

A mesura que la dificultat de les instàncies augmenta (d3, d4 i posteriors), les diferències entre solvers i heurístiques es fan molt més evidents. En general, la cerca Baseline amb Gecode degrada ràpidament, arribant a temps molt elevats o bé a *timeout* en diverses instàncies (d4, d5, d6, d9 i d10).

Impacte de les heurístiques L'heurística FF-Min mostra un impacte molt positiu en molts casos, especialment amb el solver Chuffed. Per exemple, en la instància d6, Chuffed amb FF-Min resol el problema en menys d'un segon, mentre que la resta de configuracions fan *timeout* o bé són molt més lentes. De manera similar, en instàncies difícils com d4 i d5, FF-Min amb Chuffed permet trobar solució dins el límit de temps, mentre que Gecode no ho aconsegueix.

En canvi, l'heurística IO-Min (selecció de variables en l'ordre d'entrada i assignació del valor mínim) no mostra una millora clara i consistent respecte al Baseline. En algunes instàncies mitjanes (per exemple d2 i d3) sí que redueix el temps, però en instàncies difícils sovint no evita el *timeout* i pot ser menys efectiva que FF-Min.

Comparació entre solvers En general, els resultats mostren que `Chuffed` és més robust que `Gecode` en instàncies difícils, especialment quan es combina amb heurístiques adequades. Chuffed aconsegueix resoldre instàncies que Gecode no pot, o bé ho fa en temps significativament menors. Això és especialment notable en instàncies molt restringides, on la combinació de restriccions d'adjacència, incompatibilitats i espai limitat provoca un alt grau de conflictivitat.

També és rellevant observar que Chuffed detecta alguns casos com a `UNSAT` (d9 i d10 amb FF-Min) dins del límit de temps, mentre que altres configuracions no són capaces de concloure-ho.

Conclusió A partir dels experiments realitzats, es pot concloure que l'ús d'heurístiques de cerca és essencial per obtenir un rendiment acceptable en instàncies mitjanes i difícils. Entre les opcions provades, la combinació de `Chuffed` amb l'heurística FF-Min ofereix els millors resultats globals, tant en temps de resolució com en capacitat de resoldre instàncies molt restringides dins del límit de temps establert.

3.3.2 Dades del sistema

Les proves s'han realitzat en un Ubuntu i amb un Lenovo ThinkPad T14 Gen 5 amb un Ryzen 7 PRO 8840U. No s'ha executat per threads.

4 Comentaris i reflexions

En aquesta pràctica hem constatat que, en problemes de Constraint Programming, la dificultat no depèn únicament de la mida de la graella o del nombre de finques, sinó sobretot de la *formulació* (viewpoint) i de la *estratègia de cerca*. Dos models que representen el mateix problema poden tenir comportaments radicalment diferents segons com es defineixin les variables i com es guiï el solver durant l'exploració.

Dificultats principals La primera dificultat va ser decidir com tractar les rotacions i simetries de les finques. En el model de dades inicial, descriure aquestes transformacions de manera neta i general no era trivial. Això ens va portar a explorar dos viewpoints: un que treballa amb la finca com un objecte (posició i orientació), i un altre que treballa amb les caselles individuals que formen totes les finques. El fet de tenir dos enfoques ens ha permès comparar-los.

Troballes sobre viewpoints El Viewpoint 1 (finca com a objecte amb posició i orientació) és molt expressiu per capturar rotacions i permet que el solver "decideixi" en termes més alts nivell (on col·locar cada finca i amb quina orientació). Tot i així, hem observat que és sensible a l'ordre de decisió: si el solver fixa massa aviat una orientació poc adequada, pot explorar branques molt costoses abans de detectar la contradicció.

El Viewpoint 2 (posició de cada casella de totes les finques) és més directe per expressar restriccions geomètriques, d'adjacència i de prohibició (parcs/camins). Tanmateix, incrementa el nombre de variables i reforça la importància d'utilitzar heurístiques, especialment en instàncies difícils.

Conclusió final Com a conclusió, el principal aprenentatge és que en CP no n'hi ha prou amb "tenir les restriccions correctes": cal modelar-les de manera que la propagació sigui efectiva i cal guiar la cerca perquè el solver descobreixi contradiccions aviat.